DNS Security

Assignment 2 — Build, Attack, and Defend a DNS Hierarchy

CS6008: Network Security
Department of Computer Science & Engineering, IIT Bombay

Bathula Rithvik (Roll No. 23B0939)

October 23, 2026

Contents

Overview **2**

1 Phase 1 — Building the DNS Hierarchy **2**
1.1 Design . . . 2
1.2 Verification . . . 2
1.3 Report answers . . . 2

Overview

This report documents a seven-phase study of DNS security carried out in a self-contained Docker lab. We first stand up a plain hierarchical DNS service across three isolated subnets, then mount an on-path forgery attack, add a DNSSEC chain of trust and re-run the attack, attempt a signature-stripping downgrade, walk a signed zone, move the adversary to the client's last mile, and finally encrypt that last mile with DoT. Each phase reports design choices, verification and attack evidence (terminal output and annotated packet captures), and answers to the phase's questions — not only *what* happened, but *why*.

All nodes run on isolated Docker bridge networks; nothing touches the real Internet. Every attack program is written in C/C++ with `libpcap`; `dig`, `delv`, `tshark`, and `curl` are used only to verify results. BIND 9.20.30 runs on every nameserver and on the resolver.

1. Phase 1 — Building the DNS Hierarchy

1.1 Design

The lab uses three Docker bridges — client `172.16.1.0/24`, resolver `172.16.2.0/24`, and authoritative `172.16.3.0/24` — with IP masquerade disabled on each. Docker's built-in inter-bridge isolation then drops any packet that tries to cross directly between subnets, so the *only* path between two subnets is a router container that has an interface in both. The two routers (`edge-router`, `core-router`) enable `net.ipv4.ip_forward=1` with no NAT; every other node installs static `/24` routes pointing at the router on its segment. Because a `/24` route is more specific than Docker's default route via the bridge gateway, cross-subnet traffic is forced onto our routers.

The hierarchy is a faithful miniature of the real DNS: `root-ns` is authoritative for the root and delegates `com` to `tld-ns` (glue `ns.com A 172.16.3.20`); `tld-ns` delegates `example.com` to `domain-ns` (glue `ns.example.com A 172.16.3.30`); `domain-ns` serves `A` records for `example.com` and `www.example.com` pointing at the webserver (`172.16.1.20`). The resolver is recursive with `dnssec-validation no` and uses a root-hints file that names only our private `root-ns`.

1.2 Verification

Resolution and fetch. From the client, `dig` returns the webserver address and `curl` returns the legitimate page (full log: `phase1/evidence/p1_resolve_fetch.txt`):

```
$ docker exec client dig +short example.com
172.16.1.20
$ docker exec client dig +short www.example.com
172.16.1.20
$ docker exec client curl -s http://example.com | grep -i legitimate
   <p>This is the <strong>legitimate</strong> site, served by webserver at 172.16.1.20.</p>
```

Isolation. A query sent straight to the authoritative subnet gets no answer, because the client has no route into `172.16.3.0/24` — it falls back to Docker's default gateway, which does not forward between the isolated bridges:

```
$ docker exec client dig +time=3 +tries=1 @172.16.3.10 example.com
;; communications error ... no servers could be reached
```

1.3 Report answers

(a) Root → TLD → Domain referral — what is visible on the wire. Captured on `core-router` (resolver-subnet interface) during one cold-cache resolution (`phase1/evidence/p1_referral.pcap`, decoded below):

```
1  172.16.2.10 → 172.16.3.10  query  A example.com
3  172.16.3.10 → 172.16.2.10  resp   referral: com NS ns.com, glue A 172.16.3.20
4  172.16.2.10 → 172.16.3.20  query  A example.com
6  172.16.3.20 → 172.16.2.10  resp   referral: example.com NS ns.example.com, glue A 172.16.3.30
7  172.16.2.10 → 172.16.3.30  query  A example.com
8  172.16.3.30 → 172.16.2.10  resp   ANSWER: example.com A 172.16.1.20
```

Every message is plain, unauthenticated UDP/53. On the wire an observer sees, in cleartext: the full query name (`example.com`) at every tier, the 16-bit DNS transaction ID and UDP source port that tie a reply to its query, the referral NS names and their glue addresses, and finally the answer `A 172.16.1.20`. Nothing is signed or encrypted; there is no field that binds a reply to the genuine server beyond the ID/port pair, both of which are visible to anyone on the path.

(b) How the resolver's query reaches `root-ns`. The resolver has no interface on the authoritative subnet, so it uses the installed static route:

```
$ docker exec resolver ip route get 172.16.3.10
172.16.3.10 via 172.16.2.3 dev eth0 src 172.16.2.10
```

The next hop `172.16.2.3` is `core-router`, which has a foot in both the resolver and authoritative subnets and forwards the packet (`ip_forward=1`, no NAT) out onto `172.16.3.0/24` to `root-ns`. The reply returns by the symmetric path. This is why the capture on `core-router` sees the entire referral chain.

(c) What a node on the resolver's path could do. Because the whole exchange is cleartext and unauthenticated, any node that can see and relay the resolver's traffic — exactly the position `core-router` occupies, and the position the attacker will be given in Phase 2 — can read every query and, crucially, *forge a reply*: it knows the transaction ID, the source port, and the question, so it can craft a response that the resolver will accept as genuine and cache. It can do this for one chosen name while relaying everything else untouched. We exploit precisely this in Phase 2.